# real-sw-p8-full

An AgentMPI run analysis

**Abstract**

This is the headline run of the collaborative software-development experiment: eight real agent ranks, one module each, building `minidb` — a small SQL engine graded by a 60-case acceptance suite — with interfaces published into an RMA window and cross-review enabled. It is the "less parallelizable" task in the project, and it succeeded: replaying the archived round-1 tree against the acceptance suite gives 60/60, so the population composed a working engine at the *first* integration barrier, without any agent seeing another agent's code. The trace's most important feature, however, is that the run did not know this. The harness parsed the oracle's report by slicing from the last brace in its output, which lands inside a nested per-case object and raises `JSONDecodeError`; the suite's valid 7,058-byte JSON was therefore recorded as an import failure, scattered back to all eight ranks as the definition of done, and the population spent a peer-review phase and an entire second round — 1,009.9 of 2,270.2 busy rank-seconds, about half the $1.1444 — repairing a phantom. Everything after $t = 300.8$ s in Figure 1 is the harness reacting to a report it could not read. The single thing to take away is that the population was robust to it: told falsely that its correct work had failed, it rewrote six of eight modules and still passed 60/60. A secondary result is that analysing this run turned up a defect in the analysis tooling itself: the coordination share was computed as a sum of per-invocation maxima and read 81.8 %, a figure that is not a proportion of anything. It has since been redefined and now reads 41.7 % of rank-seconds, matching the value derived by hand here to four significant figures.

## 1 What this run is

| property | value |
|---|---|
| run | `real-sw-p8-full` |
| experiment | `software` |
| campaign label | `real-sw` |
| declared world size | 8 |
| ranks appearing in the log | 8 |
| executors | `broker` × 8, `worker` × 38 |
| events | 686 |
| wall time | 499.38 s |
| job outcome | completed |

## 2 Headline measurements

| metric | value | meaning |
|---|---|---|
| achieved parallelism | 4.55× | total busy time over wall time |
| parallel efficiency | 56.8% | achieved parallelism per rank |
| idle fraction | 43.2% | rank-seconds paid for and unused |
| peak ranks busy | 8 | of 8 in the world |
| serial fraction of active time | 8.6% | time with exactly one rank busy |
| load imbalance | 1.64× | slowest rank's busy time over the mean |
| coordination share | 41.7% | rank-seconds blocked in collectives, of those available |
| coordination in flight | 78.7% | wall clock with any rank inside a collective |
| rank reattachments | 38 | fresh agent processes that took over a rank role |
| agent calls | 32 | invocations that reached a model |
| agent latency p50 / p95 | 57.77 / 190.11 s | per invocation |
| messages | 129 | 107 eager, 22 rendezvous |
| tokens sent | 133,090 | 79,354 deferred under rendezvous |
| tokens in / out | 107,118 / 54,871 | prompt and completion |
| cost | $1.1444 | 0.0209 $/kTok out |

## 3 What the analysis notices

- [warning] `neighbor_allgather/?` reports 0 messages but the fabric logged 16: the collective's own accounting disagrees with the traffic it produced.
- [warning] `neighbor_alltoall/?` reports 0 messages but the fabric logged 16: the collective's own accounting disagrees with the traffic it produced.
- [note] Load imbalance is 1.64×: the slowest rank was busy far longer than the mean.
- [ok] 38 rank reattachment(s) occurred, up to incarnation 6: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 11 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

## 4 Per-rank detail

| rank | busy (s) | occ. (%) | tasks | calls | sent | recv | tok. sent | ctx (%) | state |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 103.97 | 20.8 | 4 | 4 | 25 | 16 | 18,766 | 0.0 | finalized |
| 1 | 261.96 | 52.5 | 4 | 4 | 12 | 15 | 6,829 | 0.0 | finalized |
| 2 | 213.43 | 42.7 | 4 | 4 | 17 | 17 | 14,733 | 0.0 | finalized |
| 3 | 272.34 | 54.5 | 4 | 4 | 12 | 15 | 9,209 | 0.0 | finalized |
| 4 | 388.23 | 77.7 | 4 | 4 | 22 | 19 | 42,263 | 0.0 | finalized |
| 5 | 390.63 | 78.2 | 4 | 4 | 12 | 15 | 17,518 | 0.0 | finalized |
| 6 | 464.15 | 93.0 | 4 | 4 | 17 | 17 | 19,884 | 0.0 | finalized |
| 7 | 175.55 | 35.2 | 4 | 4 | 12 | 15 | 3,888 | 0.0 | finalized |


**Figure 1:** Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

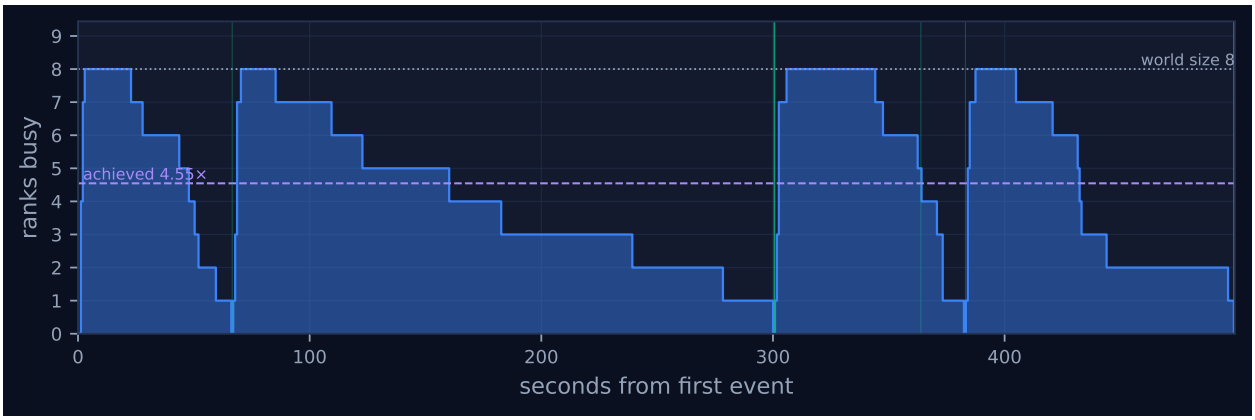


**Figure 2:** Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

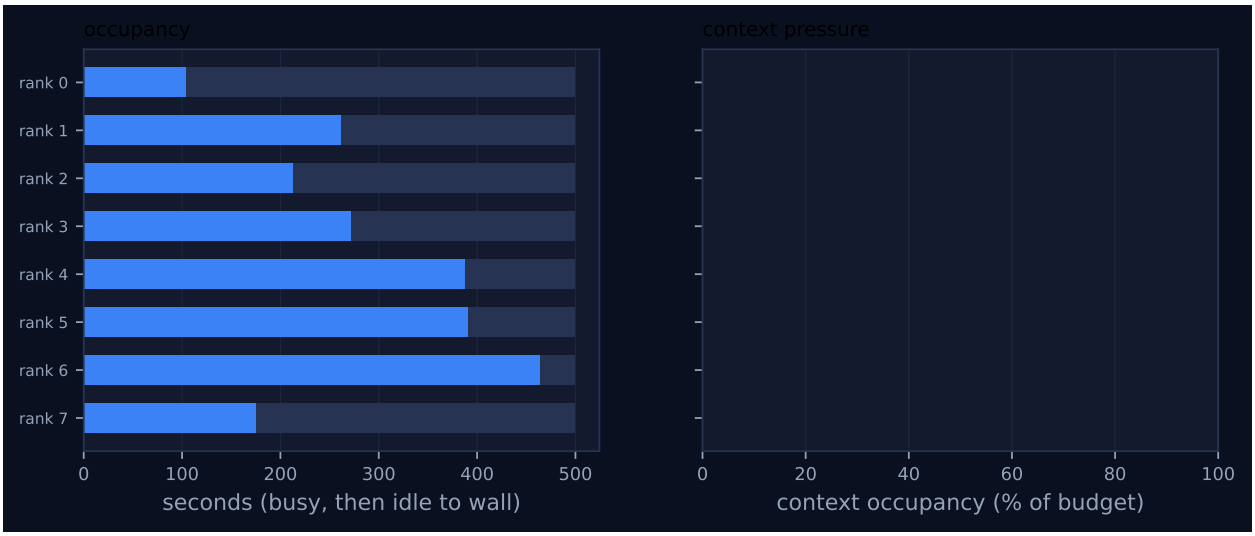


**Figure 3:** Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

# 5 Collectives, against the cost model

| op | algorithm | label | $p$ | seen | rounds | pred. | msgs | pred. | token |
|---|---|---|---|---|---|---|---|---|---|
| bcast | binomial | spec | 8 | 8 | 3 | 3 | 7 | 7 | 10,68 |
| barrier | dissemination | win_fence:interfaces-0:interfaces-published | 8 | 8 | 0 | 3 | 24 | 24 | 2 |
| barrier | central | integrate-r1 | 8 | 8 | 0 | 2 | 0 | 0 | |
| gather | binomial | collect-r1 | 8 | 8 | 3 | 3 | 7 | 7 | 29,84 |
| bcast | binomial | report-r1 | 8 | 8 | 3 | 3 | 7 | 7 | 5,62 |
| scatter | binomial | blame-r1 | 8 | 8 | 3 | 3 | 7 | 7 | 6,19 |
| neighbor_allgather | ? | review-r1 | 8 | 8 | 0 | – | 16 | – | |
| neighbor_alltoall | ? | reviewback-r1 | 8 | 8 | 0 | – | 16 | – | |
| barrier | dissemination | win_fence:interfaces-0:renegotiate-r1 | 8 | 8 | 0 | 3 | 24 | 24 | 2 |
| barrier | central | integrate-r2 | 8 | 8 | 0 | 2 | 0 | 0 | |
| gather | binomial | collect-r2 | 8 | 8 | 3 | 3 | 7 | 7 | 30,73 |
| bcast | binomial | report-r2 | 8 | 8 | 3 | 3 | 7 | 7 | 5,62 |
| scatter | binomial | blame-r2 | 8 | 8 | 3 | 3 | 7 | 7 | 6,19 |

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

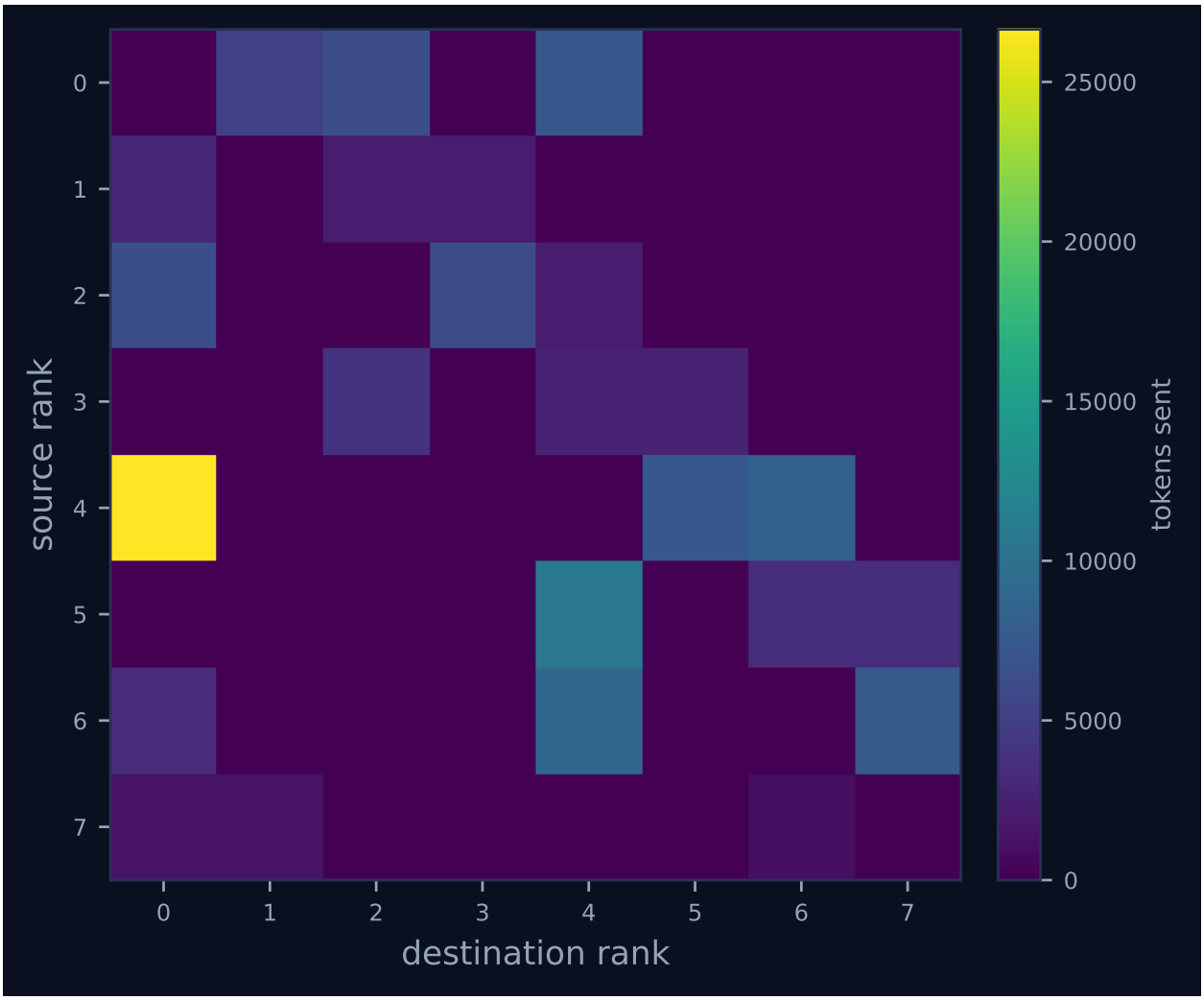


**Figure 4:** Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

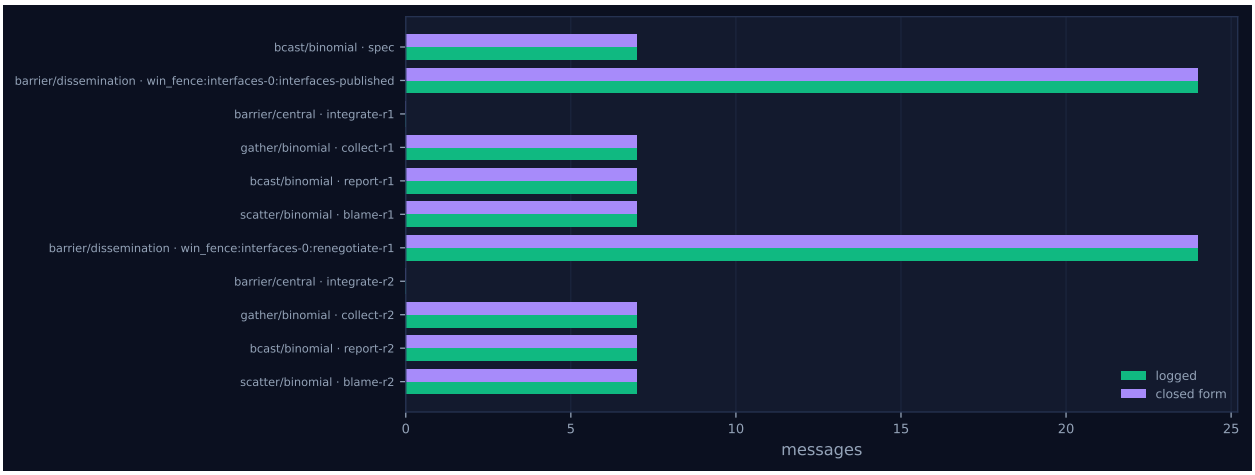


**Figure 5:** Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

## 6 Reading the timeline

Figure 1 has four bands of blue work separated by three synchronisation points, at $t \approx 67\,\mathrm{s}$, $t \approx 300\,\mathrm{s}$ and $t \approx 383\,\mathrm{s}$. They are the harness's phases: publish interfaces, implement round 1, cross-review, implement round 2. Nothing else happens; there is no lane that is busy while its neighbours are between bands, and no band that begins before the previous one has fully drained. That is the visual signature of a bulk-synchronous program, and it is by construction — every phase boundary in `experiments/software.py` is a `Window.fence` or a `barrier`.

Figure 2 is the same structure seen as a sawtooth. Each phase begins with all eight ranks busy, holds the ceiling briefly, and then decays as ranks finish and block at the next collective. Occupancy reaches the world size of 8 four times and totals 91.3 s at that level (18.3 % of wall); it sits at exactly one busy rank for 42.5 s (8.5 %, the reported serial fraction) and at zero for 4.2 s. The area under the curve is the 2,270.2 rank-seconds of work; the area between it and the dotted world-size line is the 1,724.8 rank-seconds — 43.2 % — that were paid for and unused, and essentially all of that is the decay of the four teeth. Computing per-phase efficiency from the broker claim/complete spans in the log gives 68 % for interface publication, 50 % for implement round 1, 76 % for review and 53 % for implement round 2. The two implementation phases are where the pool empties out.

The longest single wait in the run is rank 0's 215.56 s at the `integrate-r1` barrier, and it is legible as a gap in Figure 1: rank 0 finishes `impl:errors:r1` after 18.0 s of work at $t = 85.0\,\mathrm{s}$ and its lane is empty until the review band opens at $t = 301.0\,\mathrm{s}$. It was waiting for rank 6, whose `impl:engine:r1` call took 233.57 s and completed at $t = 300.4\,\mathrm{s}$; the barrier released 0.2 s later. The same pattern repeats with different stragglers: the interface fence at $t = 67\,\mathrm{s}$ waited on rank 5 (`iface:planner`, 66.5 s), the `reviewback` exchange on rank 6 (82.0 s), and `integrate-r2` on rank 4 (`impl:parser:r2`, 115.5 s). Within implement round 1 the spread between the fastest and slowest rank was 14.5× — 16.0 s for `errors` against 232.6 s for `engine`.

The amber diamonds are the window operations, and they cluster exactly where the harness puts them: eight `win.put` calls at the trailing edge of the first band, 14 `win.get` calls in a burst at $t \approx 67\,\mathrm{s}$ when each rank opens a fresh access epoch to read its dependencies, a lock/get/put/unlock quartet per rank during renegotiation between $t = 363.9\,\mathrm{s}$ and $t = 383.1\,\mathrm{s}$, and 14 more dependency reads at $t \approx 383\,\mathrm{s}$. The violet diamonds are the two `agree` invocations, at $t = 300.8\,\mathrm{s}$ and $t = 499.3\,\mathrm{s}$,

and both returned false.

Figure 4 is dominated by one cell: 26,645 tokens from rank 4 to rank 0 in four messages, a fifth of all 133,090 tokens sent in the run. That is the binomial gather fanning in — rank 4 forwards the subtree $\{4, 5, 6, 7\}$ to the root — and the two next brightest cells, $5 \to 4$ (10,622 tokens) and $6 \to 4$ (8,906), are the level below it. The plane is otherwise sparse: the review traffic forms a thin off-diagonal band consistent with a degree-2 circulant, and there is no unintended all-to-all. The right panel of Figure 3 is empty, showing 0 % context occupancy for every rank; that is not evidence of low context pressure but of the harness passing `admit=False` on every receive, so nothing was ever charged to a context account. Figure 5 shows all eleven checkable invocations agreeing exactly with their closed forms, with the two neighbourhood collectives omitted for want of a closed form.

# 7 Interpretation

### What is true by construction

The phase sequence, the module assignment and the topology are all dictated by the harness. Rank $r$ owns module $r$ because assignment is $i \bmod p$ over the eight-entry module table, so rank 0 owns `errors`, rank 1 `tokens`, rank 2 `nodes`, rank 3 `functions`, rank 4 `parser`, rank 5 `planner`, rank 6 `engine` and rank 7 `api`; the `broker.enqueue` labels in the log confirm it. The review graph is a degree-2 circulant, so 16 messages is the correct count for both neighbourhood exchanges, not a coincidence. The four agent calls per rank — interface, implement, review, implement — are the maximum the two-round configuration allows. The 38 rank reattachments, up to incarnation 6, are the broker's normal behaviour: a fresh worker process attaches for each of the 32 tasks, which is why every rank shows four or five worker incarnations, and the collectives neither noticed nor cared.

### Coordination cost, and a metric this run helped correct

An earlier version of the analysis tooling reported that collectives accounted for 81.8 % of this run's wall time, and that number was wrong in a way worth recording, because it was wrong about the instrument rather than about the run. `collective_wall_s` summed, over invocations, the *maximum* per-rank `wall_s`. For a barrier a rank's `wall_s` is its own wait, so the maximum is the wait of the rank that arrived *first*; summing those maxima adds up eight different ranks' idle intervals, each of which overlaps by construction with the other seven ranks' work, and then divides by wall time rather than by anything those intervals are a share of. One invocation, `integrate-r1`, supplied 215.56 s of the 408.28 s total, and for essentially all of it between one and seven ranks were running agent calls. The quantity was not bounded by 1 and on the translation ablations it reached 137 %.

That definition is gone. `coordination_share` is now rank-seconds blocked in primitive collectives over rank-seconds available, which cannot exceed 1, and composed invocations are excluded so that a `reduce_bcast` is no longer charged on top of the reduce and broadcast it delegates to. For this run the exclusion is a formality — all 13 invocations are primitive — and the metric now reads **41.7 %**. That is the same 1,666.2 rank-seconds I had computed by hand from the log before the fix existed, which is a useful check in both directions: the hand computation was not reading the metric it was meant to audit, and the corrected metric reproduces an independently derived number to four significant figures. A companion metric, `coordination_span_share`, answers the different question of how much of the run had coordination in flight anywhere, as the union of blocking

intervals over wall clock: **78.7 %**, or 393.0 s. My own occupancy histogram put the complement — wall time with no rank blocked — at 106.2 s, so the two agree here as well.

The two numbers together are the honest reading, and they say different things. Coordination is in flight somewhere for 78.7 % of the run because the four bulk-synchronous phases each drain gradually, so from the moment the first rank finishes until the last one does, somebody is blocked. But it costs 41.7 % of the rank-second pool, not 81.8 % of anything, and that 41.7 % is almost exactly the independently computed idle fraction of 43.2 % (1,724.8 rank-seconds): the ranks were blocked in collectives precisely when they were idle, and for the same reason. Two further measurements locate the cost. The wall time during which all eight ranks were simultaneously blocked is 0.6 s, 0.12 % of the run, so there is essentially no global stall. And the collectives that actually move data cost almost nothing: `spec`, both `collect` gathers, both `report` broadcasts, both `blame` scatters and the `review` allgather sum to 1.26 s of maximum wall between them while carrying all 133,090 tokens. What the run paid for was waiting for stragglers, not moving data or running protocol; the barrier is where the module decomposition's imbalance is collected, and the next subsection is about where that imbalance came from. The reported figure is also not understated in the way it can be on a degraded run: `coordination_is_underreported` is false here, because every collective completed and no rank blocked without living to record it.

### Load imbalance is genuine module difficulty

The 1.64× imbalance is real and it is not a scheduling artefact. Ranking the eight ranks by busy time and by the number of lines their module occupies in the round-1 tree gives a Spearman correlation of $\rho = 0.976$ — the orders differ by a single adjacent transposition, `engine` and `planner`. Rank 0 owns `errors`, 17 lines, and was busy 103.97 s; rank 6 owns `engine`, 429 lines and 37 of the 60 acceptance cases, and was busy 464.15 s. The correlation against completion tokens produced is 0.952. Dispatch cannot explain it: the interval from `broker.enqueue` to `broker.claim` across all 32 tasks has a median of 1.55 s and a maximum of 4.77 s, roughly 51 rank-seconds in total, and no task ever expired unclaimed. The imbalance is what the module decomposition bought: the harness partitioned a program into eight pieces of very unequal size and then synchronised on the largest one, twice.

### Sixteen recovery events and no failures

The 16 events carrying the `recovery` role are the two `agree` invocations, one event per rank per round. The visual vocabulary in `trace_style` maps every `ft.*` event to the fault-tolerance palette, and `ft.agree` is drawn violet along with `ft.revoke` and `ft.shrink`. Nothing was recovered from. Both invocations record `voters: [0..7]` and `excluded: []`, no rank was ever declared failed, and the run's zero `trouble` events are the correct reading of the same log. The harness uses `agree` as its *decide* step — "is the build green?" — because it is the primitive that answers that question consistently when some builders have died, not because any had. This is a taxonomy artefact of the visual vocabulary, expected for any run that uses `agree` on the happy path, and the viewer displays the tension plainly by showing "fault recovery 16" next to "FAILURES 0". One small gap is worth recording: `agree` calls `_record_collective` but emits `ft.agree` rather than a `coll.*` event, so the collectives table lists 13 invocations for a run that performed 15.

### The window: no contention, no lost updates, and a lossy renegotiation

The RMA traffic is 8 `win.create`, 16 `win.put`, 44 `win.get`, 32 `win.sync`, and 8 `win.lock`/`win.unlock` pairs. No `win.put` was flagged `stale_write`. Every renegotiation put saw version 1, overwrote version 1, and advanced its slot to version 2, so no update was lost and none was blind. The critical section was never contended — all eight `win.lock` events record `contended: false` and `waited_s: 0.0`, and `rma.contention_report` agrees with zero lock waits and zero stale writes. That is expected and it is worth being explicit about why: the window is slot-partitioned by ownership, each rank locks only the slot for the module it owns, so no two ranks can ever contend. This run therefore does not exercise the locking path at all; the `-no-locks` ablation is the only configuration that would, and it is not in this archive. The 44 reads decompose as 14 dependency reads per round (errors 0 + tokens 1 + nodes 0 + functions 1 + parser 3 + planner 2 + engine 3 + api 4) plus two per rank inside the critical section, one for the value and one for the version check.

What the renegotiation actually did is a genuine defect, and unlike the others in this run it is still present in the harness at HEAD. The critical section merges the current interface with `exports.get(m["name"])`, which is the *implementation's* export list — bare names — rather than the signature-bearing list the interface phase published. The window's contents shrank from 7,700 tokens across the eight slots at version 1 to 4,664 at version 2, a 39.4 % loss, and the per-slot `win.get` payloads show it item by item: `functions` fell from 1,375 tokens to 546, `nodes` from 1,244 to 479, `parser` from 959 to 844. The round-2 dependency reads at $t \approx 383$ s returned these degraded version-2 records. The operation is named renegotiation but is measurably a degradation: it replaces negotiated signatures with a list of identifiers.

### The oracle report never parsed, and half the run is a phantom repair

This is the run's central finding, and it inverts the headline. The in-run acceptance record is `importable: false, n_total: 0, n_passed: 0` for both rounds, and every rank received exactly one "failure" in each round — the synthetic `IMPORT` record the harness broadcasts to everyone when the tree will not import. It is false. Replaying the archived `round1` tree against the acceptance suite gives 60/60, matching the offline re-scoring stored alongside the result; replaying it against the suite as it stood at the run's commit gives 58/59, the single failure being `keywords_case_insensitive`, a case in the oracle that contradicted the specification and was itself fixed afterwards.

The mechanism is recoverable exactly from the artefacts. The stored `import_error` is byte-identical to the last 2,000 characters of the suite's own stdout, which is a single valid 7,058-byte JSON object that `json.loads` accepts today. At the run's commit, `run_acceptance` parsed it as `json.loads(out[out.rfind("{"):])`; on this output `rfind` lands at offset 6,964, the opening brace of the last per-case object, and the parse raises `JSONDecodeError: Extra data: line 1 column 93` on the trailing `]}`. The `except` branch returns the unimportable stub with the tail of stdout as the error text — which is why each rank was handed 1,200 characters of a report containing `"passed": true` and told it was an import error.

The consequences propagate through every subsequent collective. `agree` returned false at $t = 300.8$ s, which enabled the review phase; the non-empty failure list enabled the renegotiation; and round 2 ran to completion. Those two conditional phases account for 1,009.9 of the 2,270.2 busy rank-seconds (44.5 %), 59,331 of 107,118 prompt tokens (55.4 %), 27,656 of 54,871 completion tokens (50.4 %) and roughly half of the $1.1444. This is a known defect, fixed in the commit immediately following this

run, and the offline re-scoring exists precisely because of it. Two things about it are still worth stating. First, it is the reason the run's reported outcome and its true outcome disagree, and any reading of this trace that takes `agreed_green: false` at face value is wrong. Second, and more interesting, the population absorbed it: round 2 rewrote six of the eight modules and added 56 net lines, and the result still passes 60/60. Being told confidently and falsely that correct work had failed produced no regression.

### Cross-review was enabled and misdelivered

The `full` configuration is defined by shared interfaces *and* cross-review, and the second half of that did not work in this run. At the run's commit the return path was `neighbor_alltoall` over the forward review graph rather than over its transpose. The trace shows the consequence directly: `topo.graph_create` records rank 0's sources as $\{6, 7\}$, so rank 0 reviewed `engine` and `api`, while its two `reviewback` messages went to ranks 1 and 2. Checking every edge, all 16 reviewback messages were delivered to a rank whose module the sender had not reviewed. The 12,792 tokens of review that reached the round-2 prompts under the heading "peer review of your module" were, without exception, critiques of somebody else's module. The review phase cost 513.7 rank-seconds and 31,873 tokens and could not have delivered a single actionable finding to an author. This too was fixed afterwards, in the commit that added the transpose.

### The neighbourhood collectives under-report their own traffic

The two `warning` findings — `neighbor_allgather` and `neighbor_alltoall` reporting 0 messages against 16 logged — are an instrumentation gap, not a correctness problem. In this log the payload of a `coll.neighbor_allgather` event holds only `indegree`, `outdegree`, `label` and a wall time. There is no message count to read, so the analysis reports the absence as zero and correctly flags the disagreement with the fabric's tagged traffic. The traffic itself is exactly right: 16 messages each, 8 ranks $\times$ degree 2, carrying 25,396 and 12,792 tokens. The emitters were given message and token counts in a later commit. This is expected for an archived trace of this vintage; the analysis is doing its job by refusing to trust the collective's self-report.

### What this run contributes to the sweep

Against the single-agent baseline `real-sw-p1-full`, which took 1,213.0 s and \$0.4379 and needed two rounds to reach 60/60, this run reached 60/60 at the first integration barrier in 499.4 s for \$1.1444: 2.43$\times$ the speed for 2.61$\times$ the money. Against its ablation sibling `real-sw-p8-noshared`, which withheld the window, the re-scored difference is one acceptance case — 60/60 against 59/60 — with the noshared run also slower (528.6 s) and more imbalanced (2.28$\times$ against 1.64$\times$). That near-null gap is the confound the harness's own comments concede: the precise specification already pins every module's exported signatures, so it *is* a shared interface and the window has little left to contribute. The `vague-sw-*` pair exists to remove that confound. The honest contribution of this run to the ablation is therefore not the pass-rate difference but the demonstration that the mechanism works end to end — window, fence, epoch-scoped reads, locked update, agreement — on a task where eight agents genuinely had to compose.

## 8 Threats to this reading

The most serious threat is that two of the three headline behaviours in the second half of this trace are properties of the harness at a specific commit rather than of AgentMPI. The oracle-parse failure and the review misrouting were both fixed within an hour of this run, so the timeline after $t = 300\,\mathrm{s}$ tells you what the population did when misinformed and mis-reviewed, and nothing about what the protocol does when the plumbing works. Anyone comparing this run to a later one — the `vague-sw-*` pair was recorded after both fixes — is comparing across a harness change, not only across a configuration change.

The provenance stamp compounds this and should not be trusted for dating. `provenance()` shells out to `git rev-parse HEAD` when the result is *written*, not when the run starts. This result names commit `26ae006`, which was committed at 07:51:40 while the run began at 07:47:06. The sibling `real-sw-p8-noshared` names the commit that *contains* the parse fix, yet its stored `import_error` has the same signature — the tail of a valid JSON report — so it plainly ran the unfixed code too. That is fortunate for the ablation, since both arms received the same false signal, but it means the commit field cannot be used to decide which code a run executed.

Two metrics named in this document have since been corrected, and one still needs reading with care. The coordination share is discussed above: the definition this run was first analysed under could exceed 1 and has been replaced, so any older document or table quoting 81.8 % for `real-sw-p8-full` is quoting a retired metric. The zero reported for `max_claim_wait_s` is the one still to watch — it is computed over `broker.expire` events, of which there are none, so it means "no task expired unclaimed", not "no rank queued", and the real dispatch wait has to be recovered from the enqueue and claim timestamps as it is above.

The third was `tokens_deferred`, which this run's result file reported twice with different values: 212,444 from the live cost account and 79,354 from the post-hoc summariser. The difference was exact and reproducible — the live account added 79,354 tokens on the send side under rendezvous and then the full 133,090 tokens again on the receive side, because `admit=False` is passed everywhere and the receive path charged every unadmitted arrival to the same counter, so every rendezvous transfer was counted twice. The live figure consequently exceeded `tokens_sent`, which is impossible for a quantity meant to be a subset of traffic. This was a defect in AgentMPI rather than in the harness and has been fixed by splitting the field, since both quantities are worth having: `tokens_deferred` is now send-side and rendezvous-only, a subset of `tokens_sent` by construction, while `tokens_unadmitted` is receive-side and transport-agnostic, counting what arrived and was never admitted into context. `cost.summarise` derives both from the log and `job.totals()` reports both. The conflation survived as long as it did because the test that covered it asserted that a *receiver* had `tokens_deferred > 4000`, which encodes the bug as the expected behaviour; the replacement asserts across all 500 archived runs that deferred tokens never exceed sent tokens. Two things bound the damage. The inflated value reached `results/` but never reached a published number: the paper's only consumer of the field is the transport microbenchmark table, and that microbenchmark admits its receives, which is why its eager rows correctly carry no deferred tokens at all. And the log-derived figure was always right, which is why the 79,354 quoted in this document's headline table needs no revision.

One defect this document reports remains open. In the software harness, the renegotiation performed under `Window.critical` still sets its merged exports from the implementation's bare name list rather than from the signature-bearing published interface, and still costs 39.4 % of the window's content between version 1 and version 2. It is a harness defect rather than a protocol one, which is

why it has not been fixed alongside the others, but it means any run of this experiment at HEAD still degrades its own shared interfaces at the moment it claims to be renegotiating them.

Two further caveats on the numbers I have leaned on. Busy time is measured from `broker.claim` to `broker.complete` and so excludes the dispatch wait that the `agent.call` latency includes: the per-rank busy totals fall short of the summed latencies by between 5.5 s (rank 7) and 12.0 s (rank 6), 56.6 s over the run, which is the 32 enqueue-to-claim intervals. It is small enough not to disturb the $\rho = 0.976$ correlation, but it means the 43.2 % idle fraction slightly overstates true idleness and that occupancy and agent latency are not two views of the same quantity. And the 0 % context occupancy in Figure 3 is an artefact of the harness's blanket `admit=False`: this run says nothing about admission pressure, and a reader should not conclude that eight ranks exchanging 133,090 tokens fitted comfortably in a 120,000-token budget, only that nothing was ever charged against it.

Finally, this is one run of one specification with one model behind the broker, and the quantity that matters most — that the first integration barrier already produced a passing build — rests on a single sample. The executors are real (`broker` × 8, `worker` × 38, $1.1444 of measured spend, agent latencies from 18.0 s to 233.6 s), so this is evidence about agent behaviour and not only about the protocol; but the acceptance suite is 60 cases over a 130-line specification, and a suite that a first attempt passes outright is a suite that cannot discriminate between a good decomposition and a lucky one. The repair dynamics this harness was built to study remain untested here, because the only repair round it ran was answering a question that had already been answered correctly.